



Python Programming Fundamentals

Encapsulation

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. What is Encapsulation?
2. Python's Convention — The Single Underscore
3. Double Underscore — Name Mangling
4. Getters and Setters
5. Public vs Private — Design Guidelines
6. Encapsulation in Practice — Temperature



Outline

- 1. What is Encapsulation?**
2. Python's Convention — The Single Underscore
3. Double Underscore — Name Mangling
4. Getters and Setters
5. Public vs Private — Design Guidelines
6. Encapsulation in Practice — Temperature



1. What is Encapsulation?

Encapsulation means bundling data and the methods that operate on it together, while **hiding** the internal details from outside code.



1. What is Encapsulation?

Encapsulation means bundling data and the methods that operate on it together, while **hiding** the internal details from outside code.

Why encapsulate?

- Protects data from accidental modification
- Allows the class to enforce **invariants** (rules that must always hold true)
- Lets you change internal implementation without breaking external code
- Makes code easier to reason about



1. What is Encapsulation?

Encapsulation means bundling data and the methods that operate on it together, while **hiding** the internal details from outside code.

Why encapsulate?

- Protects data from accidental modification
- Allows the class to enforce **invariants** (rules that must always hold true)
- Lets you change internal implementation without breaking external code
- Makes code easier to reason about

Without encapsulation:

```
acc = BankAccount("Alice", 1000)
acc.balance = -99999      # Direct modification – bypasses all validation!
```

We need a way to **control** how attributes are accessed and modified.



Outline

1. What is Encapsulation?
- 2. Python's Convention — The Single Underscore**
3. Double Underscore — Name Mangling
4. Getters and Setters
5. Public vs Private — Design Guidelines
6. Encapsulation in Practice — Temperature



2. Python's Convention — The Single Underscore

Python does not have true private variables (unlike Java/C++). Instead, it uses **naming conventions**.

```
class BankAccount:
    def __init__(self, name, balance=0):
        self.name      = name      # public – intended for external use
        self._balance = balance    # "protected" – internal, use with caution
```



2. Python's Convention — The Single Underscore

Python does not have true private variables (unlike Java/C++). Instead, it uses **naming conventions**.

```
class BankAccount:
    def __init__(self, name, balance=0):
        self.name      = name      # public – intended for external use
        self._balance = balance    # "protected" – internal, use with caution
```

A single leading underscore `_` means: **“this is an implementation detail — don’t access it directly from outside the class.”**

It is a **convention**, not enforced by Python. Other programmers will respect it; Python itself won’t stop you.



2. Python's Convention — The Single Underscore

Python does not have true private variables (unlike Java/C++). Instead, it uses **naming conventions**.

```
class BankAccount:
    def __init__(self, name, balance=0):
        self.name      = name      # public – intended for external use
        self._balance = balance    # "protected" – internal, use with caution
```

A single leading underscore `_` means: **“this is an implementation detail — don’t access it directly from outside the class.”**

It is a **convention**, not enforced by Python. Other programmers will respect it; Python itself won’t stop you.

```
acc = BankAccount("Alice", 1000)
print(acc._balance)      # Python allows it, but it's bad form
acc._balance = -999      # This breaks the class's invariants!
```



Outline

1. What is Encapsulation?
2. Python's Convention — The Single Underscore
- 3. Double Underscore — Name Mangling**
4. Getters and Setters
5. Public vs Private — Design Guidelines
6. Encapsulation in Practice — Temperature



3. Double Underscore – Name Mangling

A double leading underscore `__` triggers **name mangling** – Python renames the attribute to `_ClassName__attribute`.

```
class BankAccount:
    def __init__(self, name, balance=0):
        self.__balance = balance      # name-mangled

    def get_balance(self):
        return self.__balance

acc = BankAccount("Alice", 1000)
print(acc.get_balance())             # 1000

# Direct access fails
# print(acc.__balance)               # AttributeError!
```



3. Double Underscore — Name Mangling

```
# But name mangling just renames it (not truly hidden)
print(acc._BankAccount__balance)    # 1000 – still accessible if you know the
name
```



3. Double Underscore — Name Mangling

A double leading underscore `__` triggers **name mangling** — Python renames the attribute to `_ClassName__attribute`.

```
class BankAccount:
    def __init__(self, name, balance=0):
        self.__balance = balance      # name-mangled

    def get_balance(self):
        return self.__balance

acc = BankAccount("Alice", 1000)
print(acc.get_balance())             # 1000

# Direct access fails
# print(acc.__balance)               # AttributeError!
```



3. Double Underscore — Name Mangling

```
# But name mangling just renames it (not truly hidden)
print(acc._BankAccount__balance)    # 1000 – still accessible if you know the
name
```

When to use __ (double underscore): when you want to prevent accidental access or name collisions in subclasses. In practice, _ (single underscore) is more common.



Outline

1. What is Encapsulation?
2. Python's Convention — The Single Underscore
3. Double Underscore — Name Mangling
- 4. Getters and Setters**
5. Public vs Private — Design Guidelines
6. Encapsulation in Practice — Temperature



4. Getters and Setters

Provide **controlled access** to private attributes via methods.

```
class Person:
    def __init__(self, name, age):
        self._name = name
        self._age = age

    def get_name(self):
        """Getter – read the name."""
        return self._name

    def set_name(self, name):
        """Setter – validate and set the name."""
        if not name.strip():
            raise ValueError("Name cannot be blank")
        self._name = name.strip().title()
```



4. Getters and Setters

```
def get_age(self):  
    return self._age
```

```
def set_age(self, age):  
    if not isinstance(age, int) or age < 0 or age > 150:  
        raise ValueError(f"Invalid age: {age}")  
    self._age = age
```

```
p = Person("alice", 25)  
print(p.get_name())      # Alice  (title-cased in setter)  
p.set_age(26)
```



Outline

1. What is Encapsulation?
2. Python's Convention — The Single Underscore
3. Double Underscore — Name Mangling
4. Getters and Setters
- 5. Public vs Private — Design Guidelines**
6. Encapsulation in Practice — Temperature



5. Public vs Private — Design Guidelines

Convention	Symbol	Meaning
Public	(none)	Part of the class's public interface — safe to use
Protected	<code>_name</code>	Internal implementation detail — avoid direct access
Name-mangled	<code>__name</code>	Strong hint not to access; prevents subclass collisions



5. Public vs Private — Design Guidelines



5. Public vs Private — Design Guidelines

Convention	Symbol	Meaning
Public	(none)	Part of the class's public interface — safe to use
Protected	<code>_name</code>	Internal implementation detail — avoid direct access
Name-mangled	<code>__name</code>	Strong hint not to access; prevents subclass collisions

Good design principle: minimise the public interface. Only expose what the user of the class **needs** to use. Everything else should be private.

```
class Stack:
    def __init__(self):
        self._items = []          # private – don't expose directly

    def push(self, item):          # public
        self._items.append(item)
```



5. Public vs Private — Design Guidelines

```
def pop(self):                # public
    if self.is_empty():
        raise IndexError("Stack is empty")
    return self._items.pop()

def peek(self):               # public
    return self._items[-1]

def is_empty(self):           # public
    return len(self._items) == 0

def size(self):               # public
    return len(self._items)
```



Outline

1. What is Encapsulation?
2. Python's Convention — The Single Underscore
3. Double Underscore — Name Mangling
4. Getters and Setters
5. Public vs Private — Design Guidelines
- 6. Encapsulation in Practice — Temperature**



6. Encapsulation in Practice – Temperature

```
class Temperature:
    """A temperature that can be read in Celsius or Fahrenheit."""

    def __init__(self, celsius):
        self._celsius = None    # initialise first
        self.set_celsius(celsius)

    def set_celsius(self, value):
        if value < -273.15:
            raise ValueError(f"Temperature {value}C below absolute zero")
        self._celsius = value

    def get_celsius(self):
        return self._celsius

    def get_fahrenheit(self):
```



6. Encapsulation in Practice — Temperature

```
    return self._celsius * 9/5 + 32

def set_fahrenheit(self, value):
    self.set_celsius((value - 32) * 5/9)

def __str__(self):
    return f"{self._celsius:.1f}C ({self.get_fahrenheit():.1f}F)"

t = Temperature(100)
print(t)                # 100.0C (212.0F)
t.set_fahrenheit(32)
print(t)                # 0.0C (32.0F)
```



Thanks for Watching - Any questions?

